# Developing Web Applications Using Symfony


In the last article in the June issue of *LINUX For You*, we discussed the development environment—the NetBeans IDE, the Symfony framework and the XAMPP Web server. In this article, let us learn more about Symfony application development, which includes the Symfony framework architecture, the folder structure of Symfony Web applications and how to develop them.

Symfony is based on the MVC architecture, with special emphasis on the abstraction of various layers. This reduces dependency among the different components. For example, at any point during development, you can change your database engine, or the HTML view of your application, with negligible impact on the schedule. We will see how this is possible.

Symfony uses an ORM (Object Relationship Mapper) and a database abstraction layer to segregate the database layer from the application layer. The ORM maps classes to database tables; queries are sent to the database via the ORM. Currently, Symfony supports two ORMs—Doctrine and Propel.

## Project folder structure

When you create a new project using the command line or NetBeans, the default directory structure (in Figure 1, *webslips* is the project's name) appears difficult to understand at first. Actually, the applications, configuration, application data, Web files, and almost everything is separated into folders; once you understand the arrangement, everything falls into place. The *'apps'* folder contains the applications; by default it has *'backend'* and *'frontend'* folders, which are the two default applications Symfony creates in a new project. Each app has its own layout (present in the *'templates'* folder). The *layout.php* file has the basic layout for the application display. Each app is also divided into smaller functional units called modules, in the *'modules'* folder (*people* in Figure 1). Each module has its actions file (PHP code for different actions) in the *'actions'* folder, and the view-related files in the *'templates'* folder. Then there is the *'web'* folder, which holds all the CSS, JavaScript and images. For example, if the *people* module has a search function, then the code for search will be added to the file *'actions.class.php'*; and the result of the function will be displayed with the template *'searchSuccess.php'*, which will be decorated with the *'layout.php'* file.

## The basics

Before diving into application design, I will introduce you to some important configuration files that define the application behaviour.

*Databases.yml*: Let's define the database for the project (the username, password and other basic details). Naturally, you should configure this first before starting development. Find the details in the *Project > Source Files > Config* folder. Here is content from a sample file:

```
all:
doctrine:
  class: sfDoctrineDatabase
  param:
    dsn:      mysql:host=localhost;dbname=webslips
    username: root
    password:
```

Here, the DSN identifies the database engine (MySQL, in this case), the host name, and the database name. The username and the password also have to be given here.